Process Concept and the Process Control Block

Lecture 5 — What Is a Running Program?

Dr. Auqib Hamid Lone
Assistant Professor in Computer Science & Engineering

PCC CS-502: Operating Systems

Department of Computer Science & Engineering
Government College of Engineering & Technology (GCET), Ganderbal

October 29, 2026

Compute. Not Click.

Welcome: One Program, Two Terminals

The big question

You compile `counter.c` into one file, `./counter`. You run it in two terminals at the same time. Each copy counts from 1 upward at its own pace. There is *one* file on disk — so what are the *two* things the operating system is running?

- The file on disk does not change while it runs; it is just bytes.
- Each run has its own counter value, its own position in the code, its own stack.
- The OS must remember all of this for each run separately.

Running thread for today

We follow `./counter` launched twice — `P1` and `P2` — and ask what the kernel must record about each.

Roadmap: Four Questions About a Running Program

1. **What is a process?** — a program in execution, not the program file.
2. **What does a process look like in memory?** — text, data, heap, and stack.
3. **What states can a process be in?** — new, ready, running, waiting, terminated.
4. **How does the kernel remember a process?** — the process control block (PCB).

Last lecture organized the kernel itself. Today we name the unit of work that the kernel manages; next lecture we watch it move between queues.

Concepts That Click

A program is a recipe written down. A process is somebody cooking it — with the oven on, ingredients half-mixed, and a finger on line 7.

Passive vs. Active

Program	**Process**
A file on disk (an executable)	A program in execution
Passive: a list of instructions	*Active*: has a program counter and live resources
One copy, however many runs	One per run, each independent
Exists until deleted	Created, runs, and terminates

- A program becomes a process when its executable is **loaded into memory**.
- Two processes may run the same program; they are still two separate execution sequences.

Silberschatz et al., Ch. 3 §3.1, pp. 101–103.

Concepts That Click

What Exactly Is a Process?

Definition

A **process** is a program in execution: the unit of work in a modern time-sharing system. It consists of the program code together with its current activity — the value of the program counter, the contents of the processor's registers, and its memory.

- Batch systems called the same thing a **job**; the textbook uses *job* and *process* almost interchangeably.
- Stallings lists equivalent views: a program in execution, an instance of a program running on a computer, the entity that can be assigned to and executed on a processor.

Silberschatz et al., Ch. 3 §3.1, p. 101; Stallings, Ch. 3 §3.1, p. 108.

Worked Example: `./counter` Launched Twice

1. Terminal A runs `./counter`. The OS creates `P1`: loads the code, allocates a stack, sets the program counter to the entry point.
2. Terminal B runs `./counter`. The OS creates `P2` from the *same* file, with a *separate* stack and data.
3. After a few seconds, `P1` has printed 812 and `P2` has printed 305.
4. The variable `count` has two values at once — one in each process's data.
5. Both program counters may point into the same loop, at different instructions.

Conclusion

Same text section, different activity. The OS tracks `P1` and `P2` as two processes.

Socratic Check 1

Question

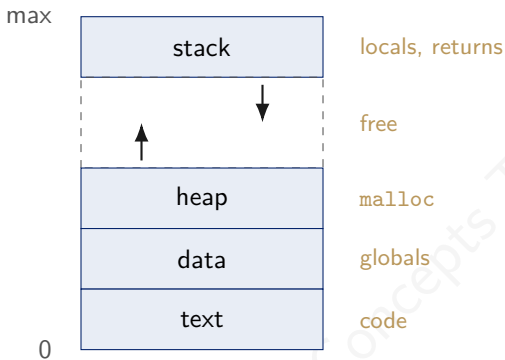
You open three browser windows of the same browser program. Is that one process or three? What would you need to know to decide?

- The number of *windows* does not decide it; the number of *execution sequences* the OS tracks does.
- One process can draw many windows; a program can also start several processes.
- The question to ask the OS is: how many PCBs exist for this program?

Before the kernel can record a process, the process needs a place to live. Its address space has four regions, each with a job.

Concepts Click

The Four Regions of an Address Space



- **Text**: the program code.
- **Data**: global variables.
- **Heap**: memory allocated at run time; grows upward.
- **Stack**: function parameters, return addresses, local variables; grows downward.

Silberschatz et al., Ch. 3 §3.1, pp. 101–102.

Concepts That Click

Worked Example: Where Does Each Variable Live?

```
int count = 0;  /* (a) */
int main(void) {
  int step = 1;  /* (b) */
  int *log =
    malloc(64);  /* (c) */
  while (1) {
    count += step; /* (d) */
    print(count);
  }
}
```

(a) `count` is global — **data**.

(b) `step` is a local of `main` — **stack**.

(c) the 64 bytes `log` points to — **heap**; the pointer `log` itself — **stack**.

(d) the instructions of the loop — **text**.

For P1 and P2

Same text; separate data, heap, and stack. `count` is 812 in one and 305 in the other.

Concepts That Click

Process Image (Stallings' View)

- Stallings calls the whole collection a **process image**:
  - user program (text),
  - user data (data and heap),
  - stack,
  - and the process's **control block**, the kernel's record of it.
- The first three are what the process uses. The last is what the *kernel* uses to manage it.

Key point

The memory regions describe what the process *is doing*; the control block describes what the kernel *knows about it*.

Stallings, Ch. 3 §3.3, p. 126.

A process is not always running. Most of the time it is waiting — for the CPU or for the outside world — and the kernel must know which.

The Five States

State	Meaning
new	The process is being created.
ready	The process is waiting to be assigned to a processor.
running	Instructions are being executed.
waiting	The process is waiting for some event (I/O completion, a signal). Stallings: *blocked*.
terminated	The process has finished execution. Stallings: *exit*.

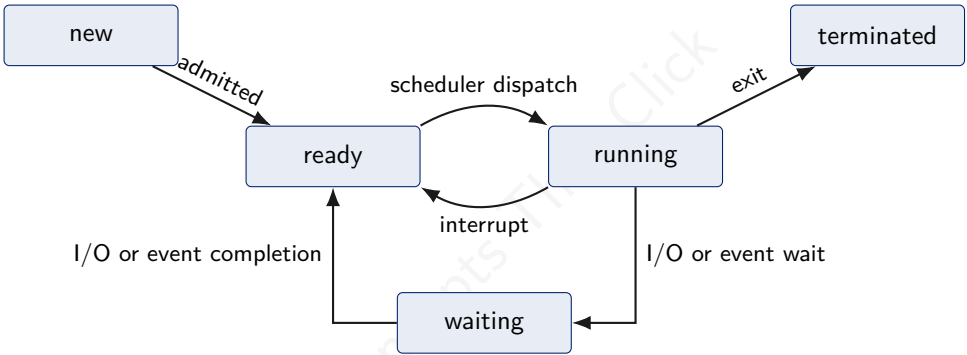
The counting rule

On one processor, **at most one** process is running at any instant; many may be ready or waiting.

Silberschatz et al., Ch. 3 §3.1, p. 103; Stallings, Ch. 3 §3.2, p. 111.

Concepts That Click

The Process-State Diagram



After Silberschatz et al., Ch. 3 §3.1, p. 103. Six transitions; each is caused by a specific event.

Reading the Diagram: What Causes Each Arrow?

Transition	Caused by
new → ready	the OS admits the process (resources allocated)
ready → running	the scheduler dispatches it to the CPU
running → ready	an interrupt, e.g. the timer: its time is up
running → waiting	it requests I/O or waits for an event (a blocking system call such as `read`)
waiting → ready	the I/O completes or the event occurs
running → terminated	it calls `exit` (or is killed)

Arrows that do not exist

waiting → running and ready → waiting. A waiting process must first become ready; a ready process is not executing, so it cannot request I/O.

Worked Trace: `P1` Reads a Key

1. `P1` is created: **new**.
2. The OS admits it: **ready**.
3. The scheduler dispatches it: **running**; it increments `count`.
4. The timer interrupt fires: back to **ready**; `P2` runs.
5. `P1` is dispatched again: **running**. It calls `read` on the keyboard: **waiting**.
6. The user presses a key; the interrupt completes the I/O: **ready**.
7. `P1` is dispatched: **running**. It calls `exit`: **terminated**.

Seven states visited, six transitions taken; the blocking `read` is the system call from Lecture 3 seen from the process's side.

Slides Only. Click

Worked Example: Counting States on One CPU

- Five processes exist. `P1` is running; `P2` and `P3` wait for the disk; `P4` and `P5` are ready.
- `P1`'s time slice expires.

After the interrupt and one dispatch

`P1` $\rightarrow$ ready. The scheduler picks `P4` $\rightarrow$ running.
Running: 1 (`P4`) Ready: 2 (`P5`, `P1`) Waiting: 2 (`P2`, `P3`).

- The total is still five: transitions move processes, they do not create them.
- With k CPUs, at most k processes run; the rest are ready or waiting.

Two-State and Five-State Models

- Stallings starts from a **two-state** model: *running* and *not running*.
- It is too coarse: a not-running process might be ready to go, or blocked on I/O — dispatching a blocked process wastes the CPU.
- Splitting *not running* into *ready* and *blocked*, and adding *new* and *exit*, gives the **five-state** model.
- Stallings later adds *suspended* states for swapped-out processes (a seven-state model); we return to these with the medium-term scheduler.

Stallings, Ch. 3 §3.2, p. 111.

Concepts Pla Tick

Socratic Check 2

Question

A process in the waiting state receives the I/O it asked for. Why does it go to *ready* instead of straight to *running*?

- Another process is probably running on the CPU right now.
- Only the scheduler decides who runs next; the I/O completion only makes the process *eligible*.
- Keeping "eligible" and "chosen" separate is what lets the OS apply a scheduling policy at all.

When P1 stops running, everything it was doing must be written down somewhere — or it can never be resumed. That somewhere is the PCB.

What Exactly Is a PCB?

Definition

The **process control block** (PCB), also called a **task control block**, is the kernel data structure that represents one process. It is the repository for any information that may vary from process to process.

- One PCB per process: `P1` and `P2` have two PCBs, even though they share one program file.
- The PCB lives in *kernel* memory; the process cannot read or write it directly.
- Creating a process means creating a PCB; terminating it means eventually freeing the PCB.

Silberschatz et al., Ch. 3 §3.1, p. 103.

Inside the PCB

PCB of `P1`

process state
process number (PID)
program counter
registers
memory limits
list of open files
...

- **State**: new, ready, running, waiting, terminated.
- **PID**: the unique process number.
- **Program counter**: the next instruction to execute.
- **Registers**: accumulators, index registers, stack pointer, general-purpose registers, condition codes.
- **Memory limits**: where its regions are.
- **Open files**: what it is using.

After Silberschatz et al., Ch. 3 §3.1, p. 103.

Concepts Tutor Clerk

The Seven Categories of PCB Information

Category	Examples
Process state	new, ready, running, waiting, halted
Program counter	address of the next instruction
CPU registers	accumulators, index registers, stack pointers, general-purpose registers, condition codes
CPU-scheduling information	priority, pointers to scheduling queues
Memory-management information	base and limit registers, page tables or segment tables
Accounting information	CPU time used, time limits, job or process numbers
I/O status information	I/O devices allocated, list of open files

Silberschatz et al., Ch. 3 §3.1, pp. 103–104.

Concordia CSAK

Why Save the PC and Registers?

- When an interrupt takes the CPU away from `P1`, the CPU's registers still hold `P1`'s values.
- `P2` is about to overwrite them.
- So the kernel copies `P1`'s PC and registers *into `P1`'s PCB* first.
- Later, it copies them back, and `P1` continues as if nothing happened.

The PCB is what makes interruption safe

State information must be saved when an interrupt occurs, to allow the process to be continued correctly afterward. Saving one and loading another is the **context switch** — the topic of the next lecture.

Worked Example: Two PCBs, One Program

Field	PCB of `P1`	PCB of `P2`
state	waiting (keyboard)	running
PID	4120	4127
program counter	inside `read` wrapper	inside the loop
`count` (in data)	812	305
open files	terminal A	terminal B
text section	shared: same `./counter` code	

Reading the table

Every row that differs is per-process information — exactly what the PCB exists to hold. The PIDs are illustrative; the OS picks them.

Stallings' Three-Part View of the PCB

Identification	Processor state	Process control
PID, parent's PID, user ID.	User-visible registers, control and status registers (PC, flags), stack pointers.	Scheduling and state, links to other processes, privileges, memory and resource ownership.

- Same content as Silberschatz's seven categories, grouped by *purpose*.
- Processor-state information is what a context switch saves and restores.

Stallings, Ch. 3 §3.3, p. 126.

A Real PCB: Linux `task_struct`

- In Linux the PCB is the **process descriptor**, a C structure of type `task_struct`.
- It holds the state, the PID, scheduling information, a pointer to the memory descriptor, the open-file table, parent and child links, and much more.
- Its `state` field records values such as `TASK_RUNNING`, `TASK_INTERRUPTIBLE`, and `TASK_UNINTERRUPTIBLE`.
- The kernel links all descriptors into a list so it can walk every process.

Note the naming

Linux uses `TASK_RUNNING` for both ready and running: the textbook's two states map to one flag plus "is it on a CPU right now?"

Bovet & Cesati, Ch. 3, pp. 79–130.

Socratic Check 3

Question

Why can the PCB not be stored in the process's own stack or heap, where it would be cheaper to reach?

- The process could then overwrite its own state, PID, or memory limits — and break protection.
- The kernel must read the PCB even when the process is not running and its memory may be swapped out.
- So PCBs live in kernel memory, reachable only in kernel mode.

A process is its address space plus its PCB. One more refinement — threads — and then the misconceptions.

Concepts · Click

Threads: A Preview

- The model so far: a process performs a *single thread of execution* — one program counter.
- Modern systems let a process have **multiple threads**, each with its own PC and registers, sharing the same address space.
- Then the PCB is extended with per-thread information.

Example

A word processor with one thread cannot type characters and run the spell checker at the same time; with two threads it can.

Silberschatz et al., Ch. 3 §3.1, pp. 104–105; threads are treated fully in Ch. 4.

Common Misconceptions

Wrong

- "A process is the program file."
- "Two runs of one program share one PCB."
- "I/O completion puts a process back on the CPU."
- "Many processes can be running on one CPU."

Right

- A process is a program *in execution*.
- One PCB per process, even for the same program.
- It goes to *ready*; the scheduler decides.
- At most one at a time per CPU; others are ready or waiting.

Takeaways

- A **process** is a program in execution; the program is passive, the process is active.
- Its address space has **text**, **data**, **heap**, and **stack**; two processes running one program share text only.
- It moves among five **states**: new, ready, running, waiting, terminated — six legal transitions, each caused by an event.
- The **PCB** records everything per-process: state, PC, registers, scheduling, memory, accounting, I/O status.
- Saving the PC and registers into the PCB is what makes interruption and resumption possible.

Concepts That Click

Next: Process States, Schedulers, and Context Switch

- Today: what a process is, how it is laid out, which states it moves through, and how the PCB records it.
- Next lecture: the queues that hold PCBs, the long-, short-, and medium-term schedulers that move them, and the context switch that saves one PCB and loads another.

Reading

Silberschatz et al., *Operating System Concepts*, Ch. 3 §3.2 [2]; Stallings, *Operating Systems: Internals and Design Principles*, Ch. 3 §3.2 and §3.4 [3]; Bovet & Cesati, Ch. 3 [1].

Further Reading (Page-Cited)

- A. Silberschatz, P. B. Galvin, G. Gagne, *Operating System Concepts*, 8e — Ch. 3 §3.1 (the process, p. 101; process state, p. 103; process control block, p. 103; threads, pp. 104–105).
- W. Stallings, *Operating Systems: Internals and Design Principles*, 6e — Ch. 3 §3.1 (what is a process, p. 108), §3.2 (process states, p. 111), §3.3 (process description, p. 126).
- D. P. Bovet & M. Cesati, *Understanding the Linux Kernel*, 3e — Ch. 3 (processes and the process descriptor, pp. 79–130), for the concrete Linux `task_struct`.

Page-verified via each book's `index.md` under `master_supporting_docs/CS502/supporting_books/`.

Copyright © Ken Slack

References I


Daniel P. Bovet and Marco Cesati.
Understanding the Linux Kernel.
O'Reilly Media, 3rd edition, 2006.


Abraham Silberschatz, Peter B. Galvin, and Greg Gagne.
Operating System Concepts.
John Wiley and Sons, 2018.


William Stallings.
Operating Systems: Internals and Design Principles.
Prentice Hall, 2001.

Concepts That Click